

MBAX 6418

Context Engineering and RAG

David Dobolyi, PhD
Assistant Professor
Stone Family Faculty Scholar
david.dobolyi@colorado.edu



Leeds School of Business
UNIVERSITY OF COLORADO **BOULDER**



Topics

- Context Engineering
- Retrieval Augmented Generation (RAG)
- Building a RAG System
- Class Exercise

Context Engineering

From Prompts to Context



tobi lutke ✓
@tobi



I really like the term “context engineering” over prompt engineering.

It describes the core skill better: the art of providing all the context for the task to be plausibly solvable by the LLM.

9:01 PM · Jun 18, 2025 · **2M** Views

346 1.1K 8.5K 2.3K



Andrej Karpathy ✓
@karpathy



+1 for "context engineering" over "prompt engineering".

People associate prompts with short task descriptions you'd give an LLM in your day-to-day use. When in every industrial-strength LLM app, context engineering is the delicate art and science of filling the context window with just the right information for the next step. Science because doing this right involves task descriptions and explanations, few shot examples, RAG, related (possibly multimodal) data, tools, state and history, compacting... Too little or of the wrong form and the LLM doesn't have the right context for optimal performance. Too much or too irrelevant and the LLM costs might go up and performance might come down. Doing this well is highly non-trivial. And art because of the guiding intuition around LLM psychology of people spirits.

On top of context engineering itself, an LLM app has to:

- break up problems just right into control flows
- pack the context windows just right
- dispatch calls to LLMs of the right kind and capability
- handle generation-verification UIUX flows
- a lot more - guardrails, security, evals, parallelism, prefetching, ...

So context engineering is just one small piece of an emerging thick layer of non-trivial software that coordinates individual LLM calls (and a lot more) into full LLM apps. The term "ChatGPT wrapper" is tired and really, really wrong.



tobi lutke ✓
@tobi · Jun 18, 2025

I really like the term “context engineering” over prompt engineering.

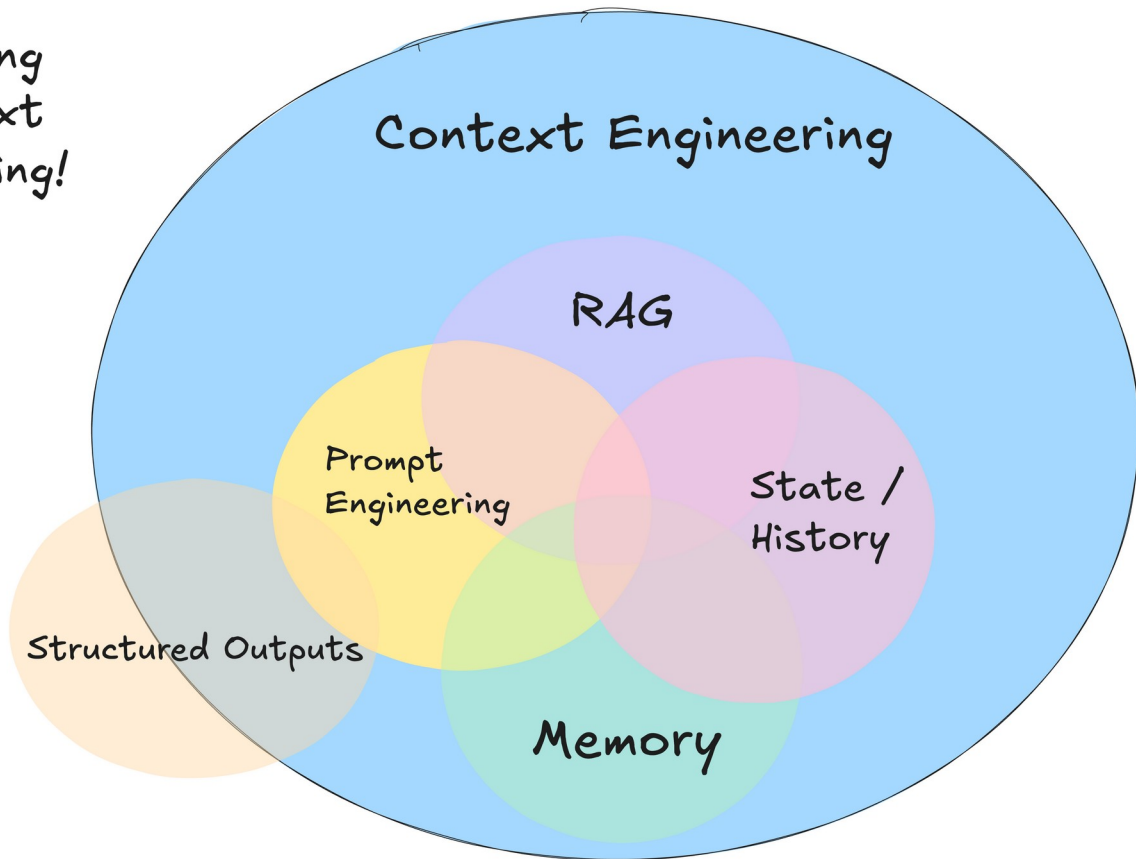
It describes the core skill better: the art of providing all the context for the task to be plausibly solvable by the LLM.

9:54 AM · Jun 25, 2025 · **2.3M** Views

531 2.6K 14K 9.1K

But What Is Context Engineering?

Everything
is Context
Engineering!

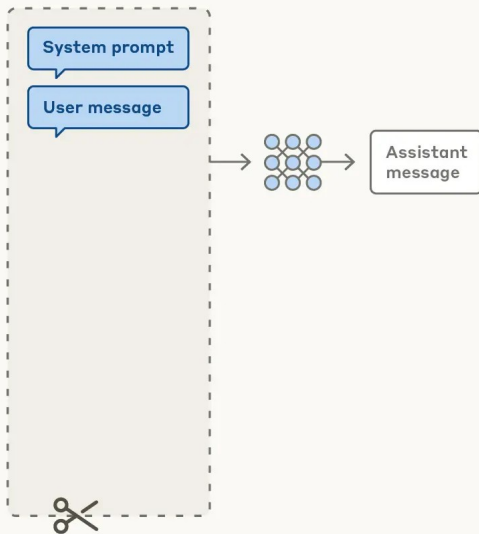


Expanding the Scope

Prompt engineering vs. context engineering

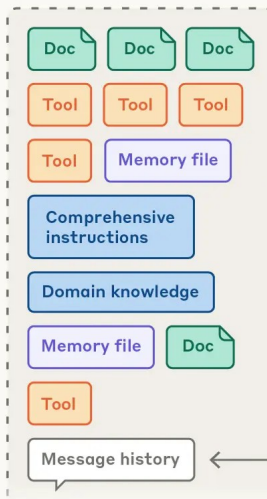
Prompt engineering
for single turn queries

Context window



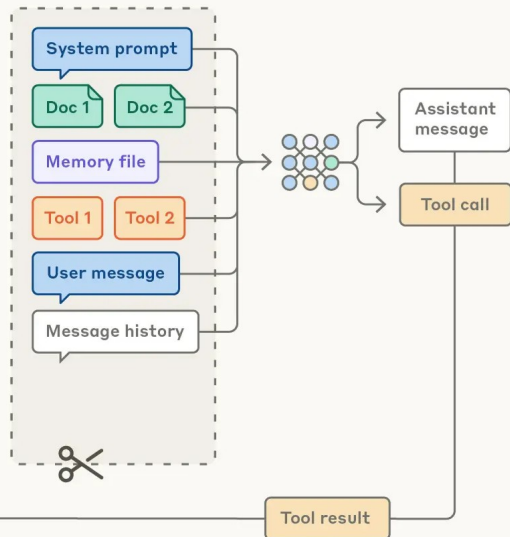
Context engineering for agents

Possible context to give model



Curation

Context window



Context Location Matters?

- Historically, models suffered from a **Lost in the Middle (LITM)** effect: LLM accuracy dropped when relevant facts occurred in the middle of input context ([Liu et al. 2023](#))
- Some newer models are more resistant to this effect, but performance still depends on the model, task, and/or surrounding content (e.g., [McKinnon 2025](#); [Zhang et al. 2026](#))

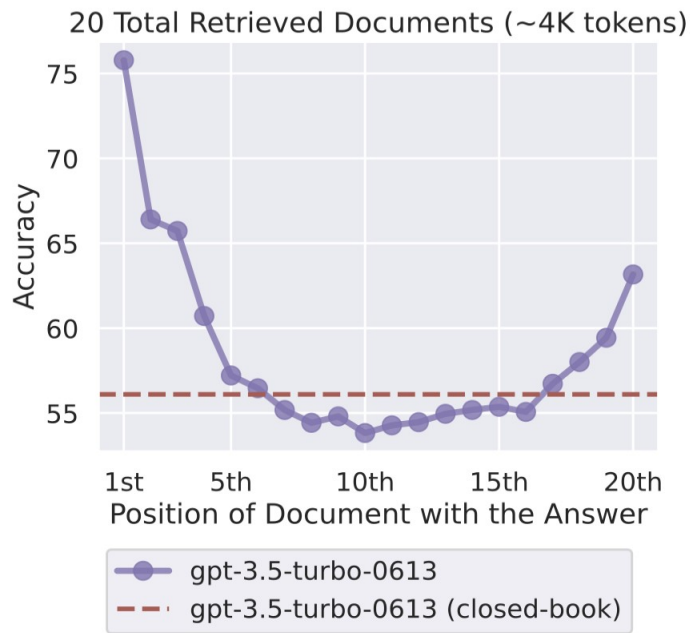
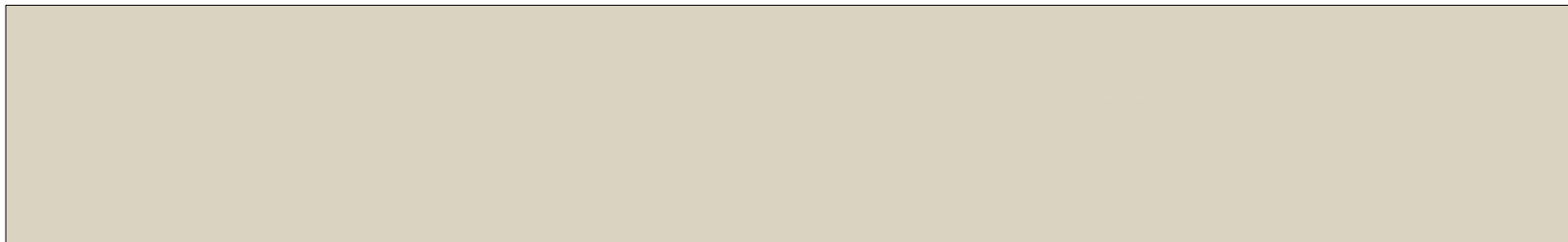


Figure 1: Changing the location of relevant information (in this case, the position of the passage that answers an input question) within the language model’s input context results in a U-shaped performance curve—models are better at using relevant information that occurs at the very beginning (primacy bias) or end of its input context (recency bias), and performance degrades significantly when models must access and use information located in the middle of its input context.

Context Is Not Infinite

- Even very large context windows can fill up during long-running or complex tasks
- Older information may need to be removed or summarized
- Agent harnesses (e.g., Hermes) can manage this for you automatically

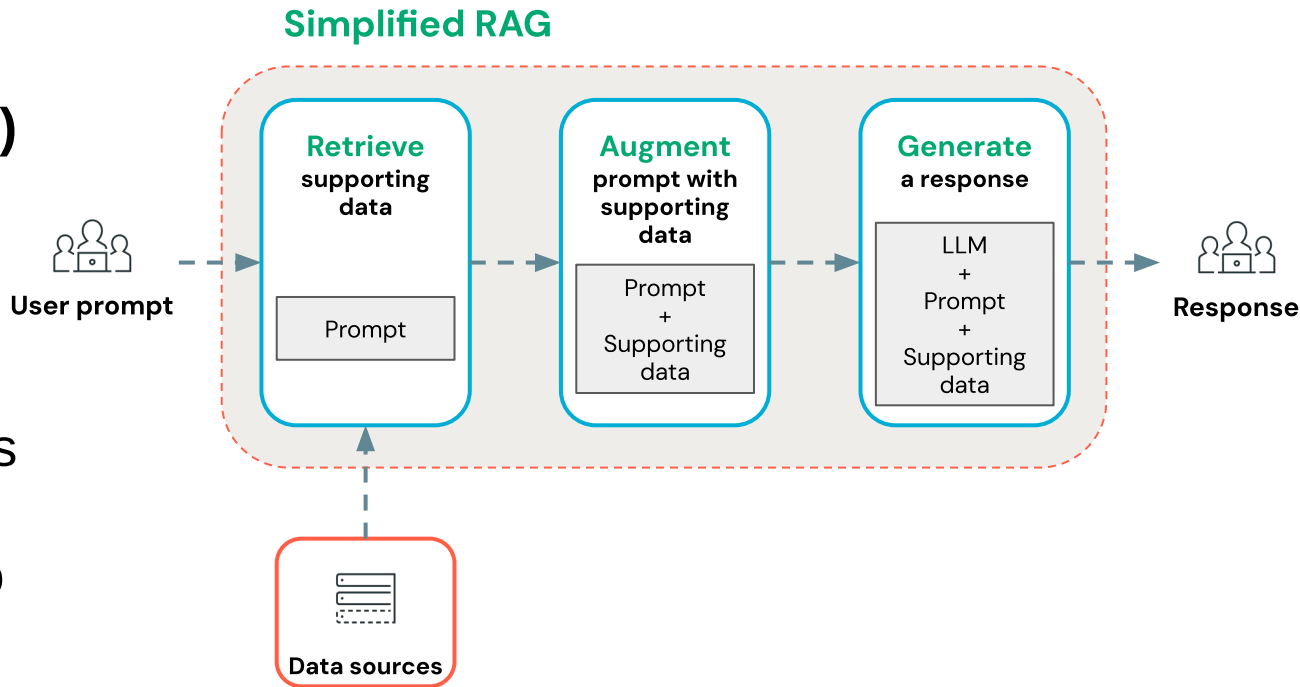


- But what about context engineering for apps you create?

Retrieval Augmented Generation (RAG)

What Is RAG?

- **Retrieval-augmented generation (RAG)** retrieves relevant information from documents, databases, and other data sources and adds it to a model's context to help generate a response to a user's prompt/question



Why Use RAG?

- RAG gives a model access to information beyond its training data, including more recent or private information
- It selects relevant material from a large collection – or **corpus** – and grounds responses in sources, making answers easier to verify, potentially reducing hallucinations

Benefits of RAG

Grounding

Links answers to real evidence instead of relying only on memorized training data. This reduces hallucinations by tying outputs to verifiable sources.



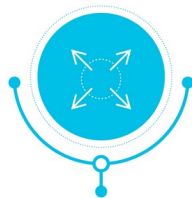
Cost efficiency

Cuts down on repeated fine-tuning and heavy compute cycles. Shifting knowledge into external systems lowers overall costs.

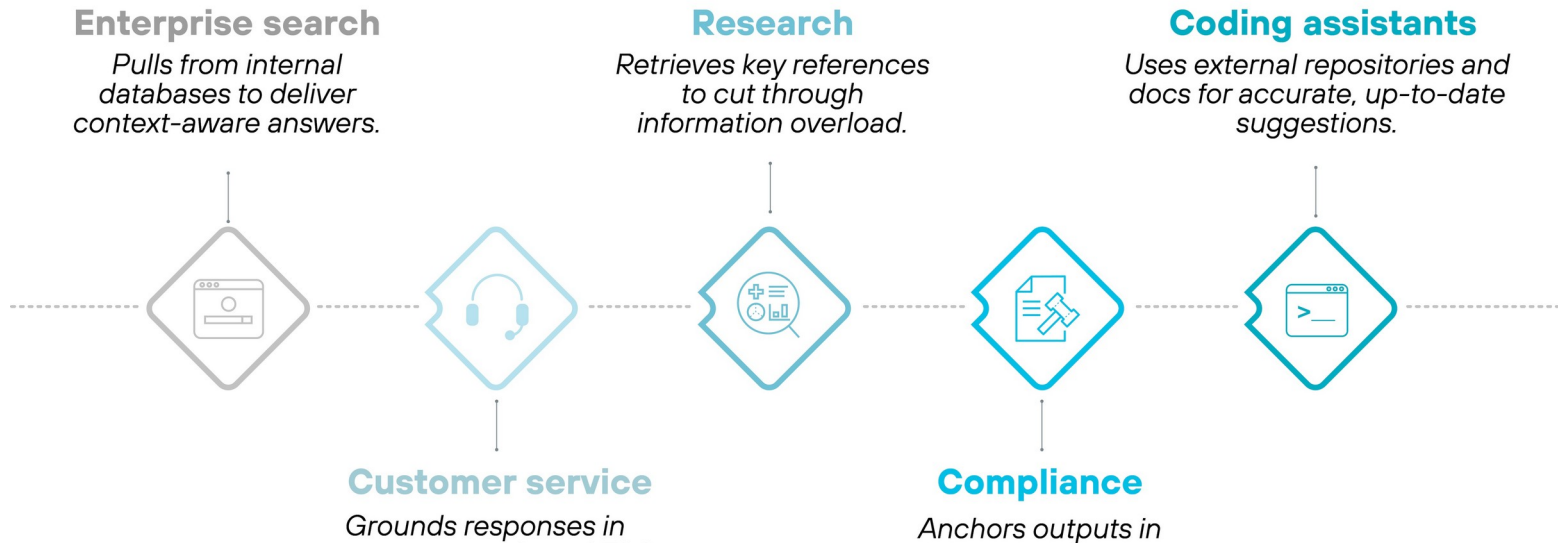


Scalability

Lets one model adapt to many domains without constant retraining. Retrieval keeps knowledge bases current, making it easier to scale across use cases.

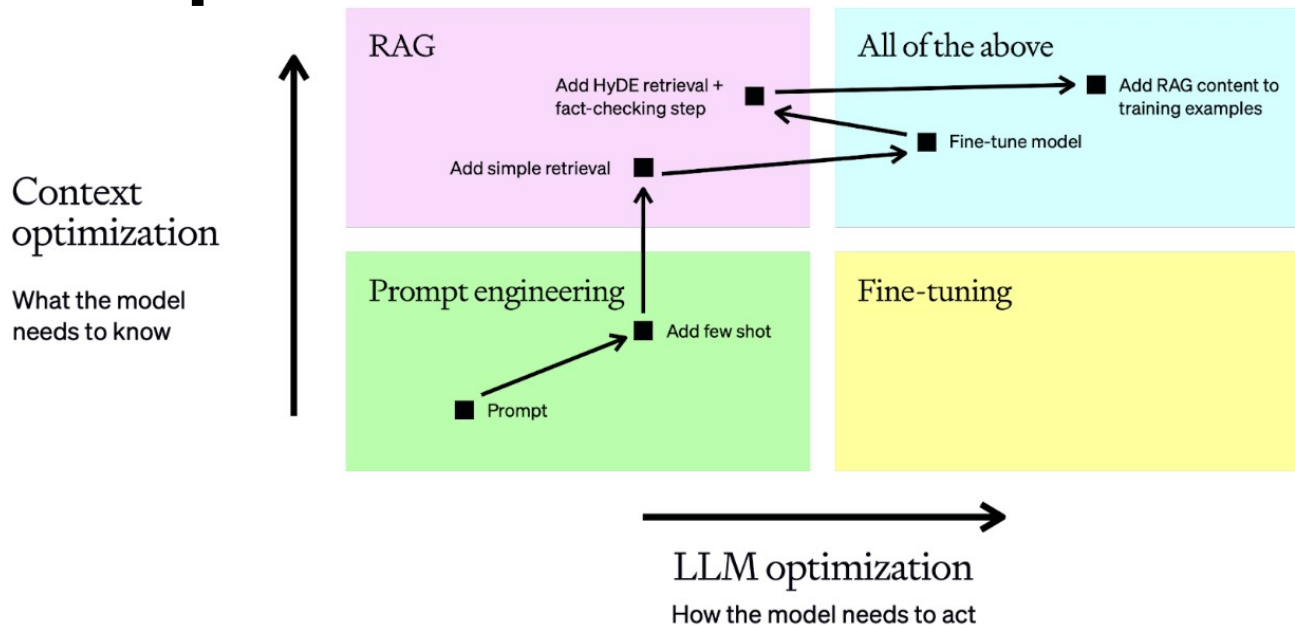


RAG Use Cases



- RAG lets your app draw on codebases, course materials, private documents, and other information sources to support a variety of tasks

Context Optimization

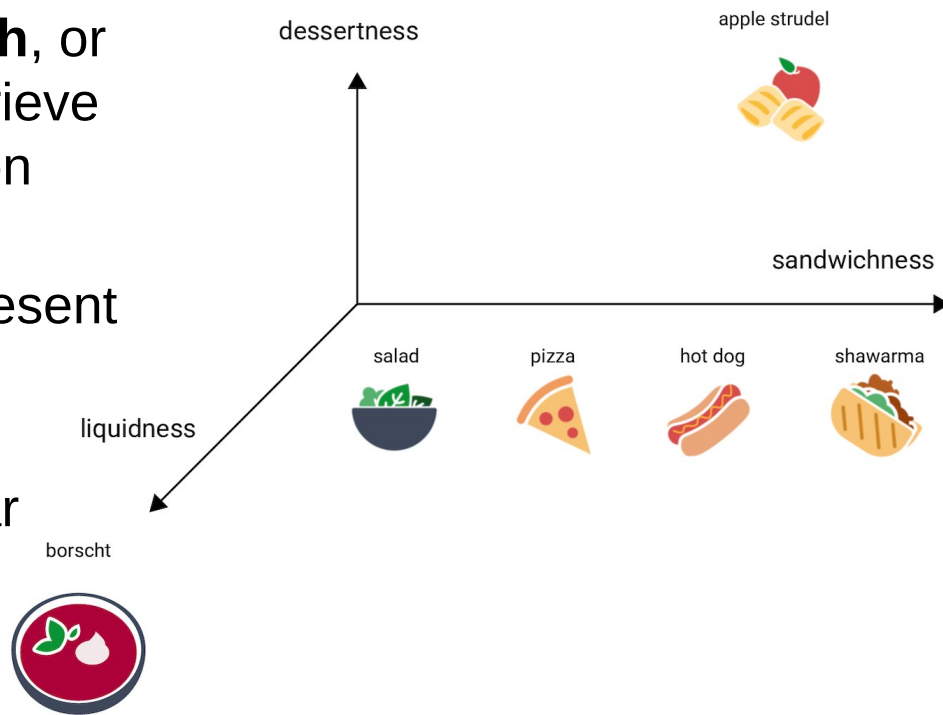


- RAG optimizes context by supplying relevant information when the model responds
- **Fine-tuning** adapts the model itself through additional training—a topic we'll cover later

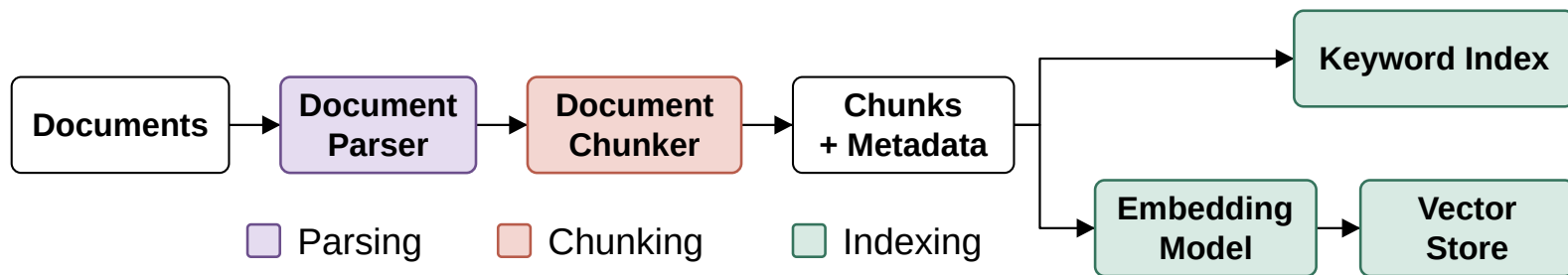
Building a RAG System

Semantic Search and Embeddings

- RAG often uses **semantic search**, or search based on meaning, to retrieve info relevant to the user's question
- Semantic search typically uses **embeddings** to numerically represent text, images, and other data
- Embeddings are like coordinates in high-dimensional space: similar items tend to lie closer together
- **Vector search** compares the question's embedding with data embeddings to find potential matches



Preparing Documents (Docs) for Search



- Docs added or updated in the RAG corpus must be prepared for search
- **Parsing** extracts text and structure from each doc
- **Chunking** divides the parsed content of each doc into useful passages while preserving source metadata (e.g., doc name, page number)
- **Indexing** prepares the chunked passages for both:
 - Keyword (lexical) search to find chunks containing matching words
 - Vector search, which uses embeddings to find chunks with similar meanings

Chunking: How Much Context Is Enough?

1 Find Sections

2 Create Chunks

3 Keep Source Details

```
course-guide.md EXAMPLE
# Course Guide

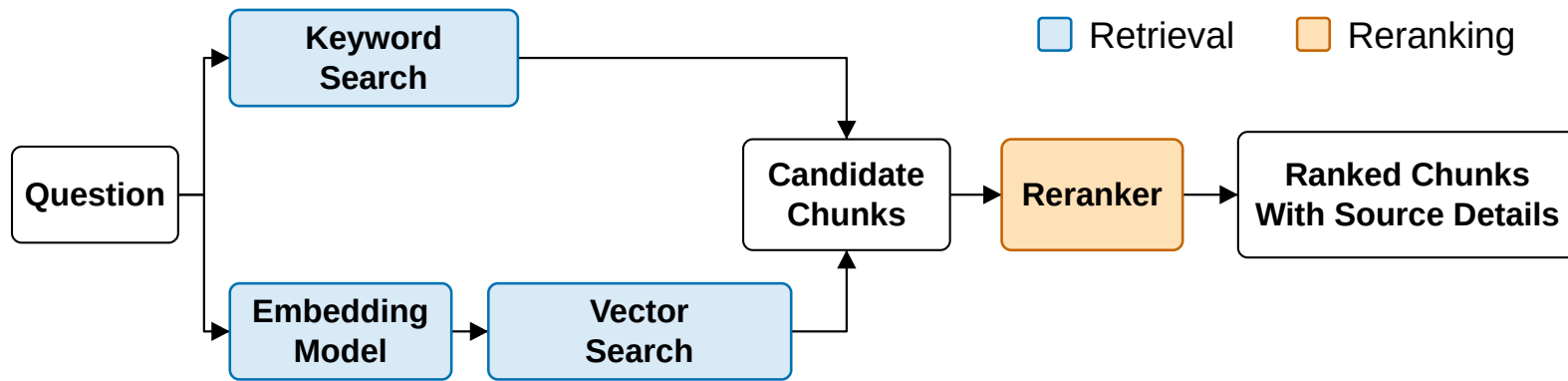
## Project Submissions
Include your code and a short explanation.

## Working Together
Discuss ideas and credit shared work.

## Getting Help
Bring your question and what you tried.
```

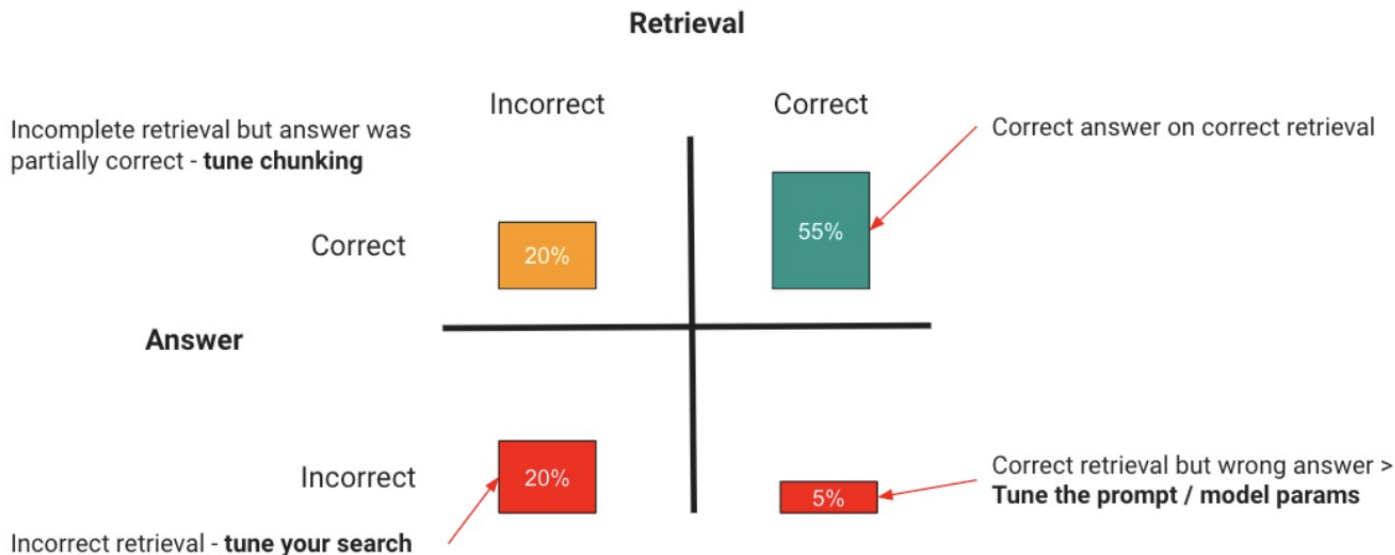
- Each chunk should contain just enough context to be useful in isolation
- Common chunking strategies:
 - **Fixed size** – chunks with a set size, often with overlap
 - **Recursive** – split at boundaries like paragraphs, then divide oversized chunks at finer boundaries like sentences
 - **Document-based** – keep related information together based on structural boundaries such as headings (see example above)
 - **Semantic** – split based on changing topics or meanings

Hybrid RAG Pipeline with Reranking



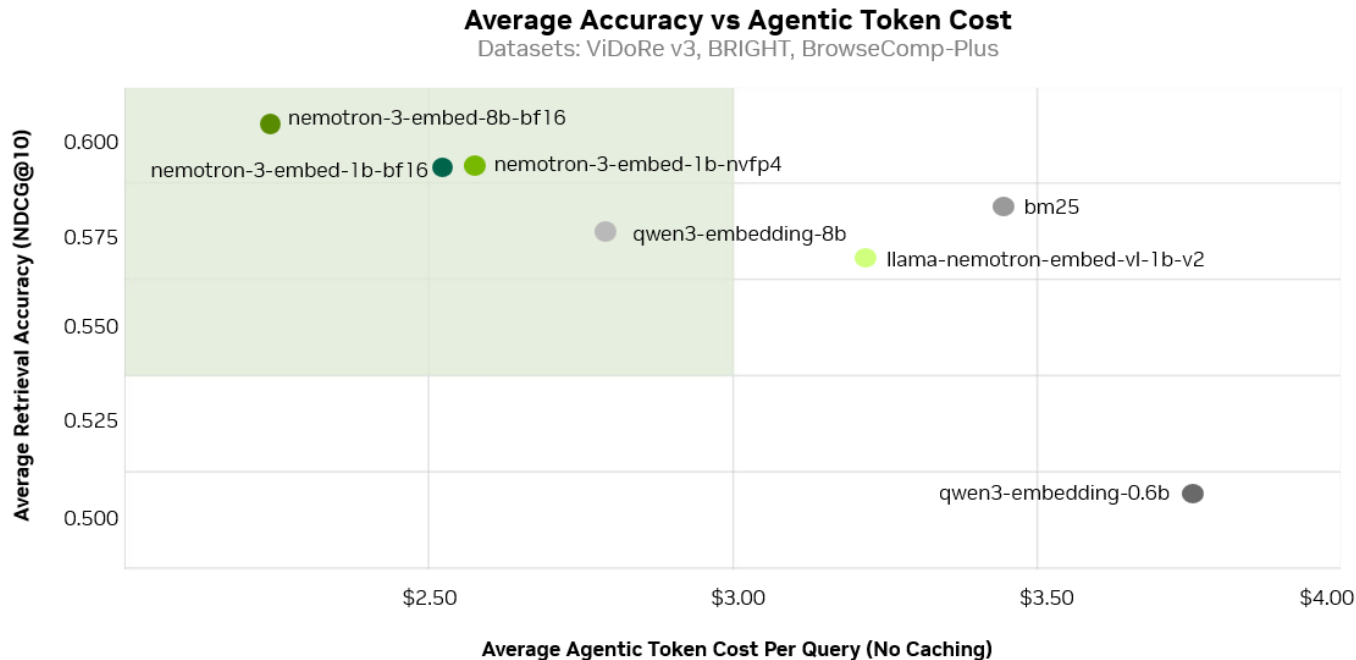
- RAG uses information from prepared documents to help an LLM answer the user's question
- **Retrieval** finds relevant *candidate chunks* using both keyword search and vector search in **Hybrid RAG**
- **Reranking** compares these candidates to the original question and prioritizes which chunks are supplied to the LLM as context

Accurate RAG is Challenging



- Accurate RAG requires checking both *retrieval quality* and *answer accuracy* using questions relevant to your app
- Use the [MTEB Leaderboard](#) to shortlist retrieval models and then test them on your own documents and questions

RAG Done Right Can Be Valuable



- Retrieving relevant evidence early can reduce tokens spent processing irrelevant information, repeating searches, and performing unnecessary reasoning – and this can save money, improve efficiency, etc.

Class Exercise